Hierarchical Agent Budgets Under Uncertain Model Usage

OpenBot

***Abstract*—A model call spends money before its full cost is known. Several agents may also share intersecting member, task, and organization caps. This design and analysis paper separates confirmed charges, active reservations, and unknown cost exposure. It checks every cap while sharing admission opportunities through a weighted tree. Estimated task value ranks model choices within each share. We define an estimate-and-release baseline and derive a late-receipt counterexample. The proposed scheduler retains unresolved exposure. We prove a spending invariant under finite cost bounds and a conditional risk bound for uncertain costs. Neither follows from a rough token estimate alone. A finite experiment enumerates 20 event orders and 25 cost pairs for each of two accounting policies. In the 180 cases per policy whose costs fit their holds, releasing unknown holds exceeds a cap in 10 cases and retention in none. Outlier cases test a deterministic overrun bound. These results check the stated model; they do not establish provider cost bounds, product conformity, or performance of the full scheduler.**

***Index Terms*—Agent scheduling, Budget admission, Uncertain cost, Fair allocation**

I. Introduction

Several agents can each see enough money for a next step, yet together commit more than the team can pay. A missing receipt makes the problem harder: work may cost money before the application can name the amount. A correct bill after work ends does not make the earlier decision sound.

The unit of admission is one provider call. A task can change its plan after each result. Calls have different sizes, prices, and chances of helping. Slowing text delivery need not stop the provider from having already made and billed that text.

We define a baseline that admits cost estimates and releases unknown reservations. We then propose retained exposure and weighted sharing.

The contribution is a joint accounting design for delayed model usage: cost envelopes, saved prices, intersecting caps, and a limited sharing guarantee. Retaining uncertain money is an established escrow rule, not a new principle. Delegation divides an existing share. Hard spending bounds need finite call costs; probability bounds need valid conditional coverage. Fair spending does not imply equal task success.

This paper gives definitions, conditional proofs, and completed finite experiments. It assumes durable receipt storage. The full scheduling study remains a proposal. The finite model supplies explicit failure traces. It makes no claim about a deployed implementation.

II. An Estimate-and-Release Baseline

We define the following baseline for analysis. Its state changes are a complete comparison method, not an observation about a deployed service. Money is an integer count of micro-USD; one dollar is one million units. Each call has an immutable identifier, start time, owner, actor, access key, agent, and task. An access key identifies a caller, while model tokens are billed units of input or output. A call can meet several budget caps. Shared-provider credits apply only to calls funded by that shared account.

A price schedule maps start time and total input size to five rates: uncached input, cache reads, cache writes, output text, and reasoning output. Rates are micro-USD per million tokens. The selected input tier sets rates for the whole call. Cost is the sum of token counts times their rates, divided by one million and rounded up. The baseline saves the entire schedule at admission. Confirmation selects the tier using actual input size and the saved start time. A later price change does not change that call.

The baseline estimates text input as the ceiling of character count divided by four. Each non-text part adds 256 estimated input tokens. It estimates 1,024 output tokens, zero reasoning tokens, and no cache use. These are chosen heuristics, not claimed token bounds. The saved schedule prices this vector to produce reservation r. Missing prices yield recorded cost zero in this permissive baseline, without implying a free external invoice.

For every cap, write B for its limit, C for confirmed cost, and H for pending reservations. Admission atomically checks $C + H + r \leq B$ for every matching cap and inserts the reservation only if all pass. A pending call becomes confirmed when a complete receipt arrives: subtract r from H and add its cost c to C. Repeating the same receipt changes nothing. Conflicting counts after confirmation cause an error without changing totals.

When usage is missing, the baseline marks a pending call unknown and subtracts r from H. Repeating that notification changes nothing. Its public cost remains unknown, not zero. A later receipt changes it to confirmed and adds c to C. No new budget check can undo this incurred charge. For both baseline and proposal, calls belong to the budget period in which they were admitted; ending a period does not erase its records.

A counterexample follows directly. Let $B = 1,000$ and initially $C = H = 0$. Admit a call with $r = 1,000$, mark it unknown, then admit and confirm a second call costing 1,000. Confirm the first with the same cost. Each admission passed, yet final cost is 2,000. All events lie within one period.

This failure needs neither a race nor an inaccurate estimate: releasing unresolved exposure is sufficient.

TABLE I
ACCOUNTING TRANSITIONS. THE PROPOSED UNKNOWN HOLD U REMAINS UNAVAILABLE FOR NEW CALLS.

Event	Baseline change	Retained exposure
Admit	Add r to H	Add r to H
Pending receipt	$H -= r$; $C += c$	$H -= r$; $C += c$
Missing usage	Remove r from H	Move r: H to U
Late receipt	Add c to C	$U -= r$; $C += c$
Same receipt again	No change	No change

III. Problem Model and Assumptions

Let a candidate call be j. It names a task, a provider and model option, the paying owner, the original actor and access key, and its delegation path. Let $G(j)$ be the set of spending constraints that apply to it. This set includes task ancestors and all matching organization, member, agent, and access-key caps. A call counts once in each distinct constraint. Counting it at a task and its parent is deliberate; adding it twice to one parent is not.

These constraints do not form one tree. A member may use several agents, and one agent may serve several members. A member cap and an agent cap overlap. We therefore use a set of constraint memberships for safety and a separate policy tree for sharing. A possible sharing tree is organization, then member, then root task, then delegated work. Creating another child divides the root task's existing weight. It does not add weight at the member level.

For each constraint g, let B_g be its money limit. Let C_g be confirmed cost, H_g be holds for active calls, and U_g be holds for calls with unknown cost. Define exposure and available room by

$$E_g = C_g + H_g + U_g, \quad h_g = B_g - E_g. \tag{1}$$

An exposure hold is a scheduling fact, not a charge. The scheduler may say that 20,000 micro-USD is unavailable because a receipt is missing. It must not say that this amount was spent.

Every call has a saved start price schedule p_j and a hold r_j. In strict mode, the adapter supplies a finite set or region of possible billed token counts T_j. The hold is the largest rounded cost under that schedule over T_j. This requires bounds on all billed classes, including hidden reasoning and cache writes. An output limit alone may not bound every charge. If a provider gives no enforceable finite bound, strict admission is unavailable for that option.

In risk mode, the hold is an estimated upper cost with an assigned failure probability α_j. This value is meaningful only if its coverage is valid given the full history before admission. A percentile fitted to a convenient old sample is not such a guarantee. The scheduler records whether a hold comes from an enforced bound, a tested statistical model, or an unsupported estimate.

The proof model uses budget cohorts fixed at admission. A call that crosses midnight stays in its original cohort. Old unknown holds remain there until resolved; they also remain in an all-time credit or liability constraint when one applies. A period reset alone cannot erase a possible debt.

We assume accurate attribution, atomic updates to all matching constraints, stable integer money arithmetic within its checked range, and no hidden call that bypasses admission. The spend proof concerns the saved accounting schedule. It covers the outside invoice only if that schedule and token bound also bound the provider's true charges. Currency changes, fees, and later invoice adjustments require their own bounds.

IV. Related Work

O'Neil's escrow transaction method reserves part of a shared amount while a remote transaction may still complete. It explicitly keeps that amount tied up when the remote outcome is unknown. [1] Retaining an unknown call hold follows this prior rule. The contribution sought here is the combination with provider cost envelopes, saved prices, intersecting constraints, and later sharing corrections. A full evaluation must compare with a plain escrow ledger using first-come admission, not only with a baseline that drops uncertainty.

Bounded counters extend escrow rights to preserve numeric limits across replicas. [2] Our model instead assumes atomic updates across all caps touched by a call. Rights partitioning could reduce coordination in a later implementation, but safe counter updates do not bound an outside bill. The provider cost envelope remains a separate premise. We claim neither a new distributed counter nor a coordination-free algorithm.

Deficit round robin gives variable-sized work a share by adding a service allowance to each queue and carrying unused allowance forward [3]. We use that simple idea for money reservations. A provider call differs from a packet because its true size may be learned much later. The proposed method therefore makes fairness in reserved money explicit and tracks the gap from confirmed spending.

Hierarchical fair service curves study a tree that shares a link while serving timing goals [4]. They expose conflicts between exact sharing and real-time service. Our money limits are finite stocks. We separate hierarchy from service policy but claim no delay guarantee. An empty task cap can block one queue while another runs.

Dominant Resource Fairness treats users whose work needs several resource types [5]. Its dominant share makes heterogeneous resource demand comparable. Money and local token rate are tempting inputs to such a rule. Yet a set of local buckets has no common cluster capacity, and model costs are uncertain. We therefore do not call our rule dominant-resource fair or inherit its strategic properties. Intersecting money caps remain admission constraints; node-local rate gates remain separate.

Kelly, Maulloo, and Tan connect network rate allocation with utility and proportional fairness [6]. Their framework explains why an allocation goal must be stated, rather than hidden inside a rate limiter. Here we choose weighted oppor-

tunity in money, then use estimated task value within each owner's share. We do not claim that this discrete policy solves a continuous utility optimum.

Bertsimas and Sim study the tradeoff between protecting against uncertain coefficients and losing nominal value [7]. Our strict and risk modes expose that tradeoff at the call boundary. We use full individual upper holds in strict mode. We do not import their uncertainty-budget theorem into an unbounded provider-cost setting.

Bandits with Knapsacks joins learning about rewards with resource limits [8]. It is a useful basis for future model-option learning. Our design first fixes how missing receipts affect available money. It offers no regret bound: task values may change with previous answers, feedback can be delayed, and receipts may never arrive. Finally, RFC 2697 gives a precise token-bucket marker [9]. It helps separate a traffic envelope from money allocation. The local debt-based gate defined below differs from that marker.

V. Admission and Allocation Method

A. Price and Cost Envelopes

At dispatch preparation, the scheduler reads the applicable price schedule and a provider capability description. It forms a token envelope from the exact or bounded input, maximum output, reasoning limit, and possible cache treatment. It evaluates each reachable input tier. This matters when crossing one input threshold raises the rate for all tokens. Taking a high token count at the estimated low tier is not a valid upper bound.

The call stores the schedule, envelope source, estimate version, and rounding rule. A new price changes later calls only. Waiting across a price window requires a new hold calculation. Missing prices block strict admission. An owner may assign a provider to a separate outside budget, but the dollar bound then excludes that invoice.

Expected cost differs from the admission hold. A call with expected cost 5,000 and an upper hold of 20,000 uses 20,000 of safety room. Expected cost may inform value ranking but cannot replace a strict hold. Each retry that may produce a separate charge needs a new exposure entry.

B. Transactional Admission Across Caps

A candidate can start only if every g in $G(j)$ has $E_g + r_j \leq B_g$. All reads, checks, and hold inserts share one transaction. Lock every affected cap in a fixed order. The transaction includes shared credit and cross-organization caps when they apply. A lock limited to one organization is insufficient for a constraint shared across organizations.

After admission, active holds move to unknown holds when a receipt is missing. This move does not change exposure. A confirmed receipt of cost c_j removes the hold and adds that cost. A trusted final statement that no charge exists removes the hold without a charge. A timeout alone is not that statement. An owner may accept an unresolved loss and fund it from a separate risk account, but the liability still counts against that account.

If a risk-mode receipt costs more than its hold, the scheduler records the full cost and stops further admissions against any exceeded cap. It does not clip the bill to preserve a nice invariant. If a limit is lowered below existing exposure, the same stop rule applies. The strict proof assumes no such downward limit change while it claims the old bound. Users can always stop new work; they cannot use a new cap to undo work already sent.

C. A Tree of Money Shares

Each sibling group in the policy tree has fixed positive weights. The scheduler visits siblings in a stable round order. A child is eligible when some queued option fits its money caps, before checking deficits. Each round gives it allowance Qw_i, where Q is the base quantum and w_i is its weight. Its deficit balance carries unused allowance. A call must fit the deficit at every sharing ancestor as well as every spending constraint. Admission subtracts the same hold from those ancestor balances.

At each leaf, the scheduler ranks a finite set of useful next actions. Options can include a smaller output limit, a cheaper model, waiting, or ending the task. The default score is expected gain in task value divided by the hold. Zero-cost options rank directly by value. A visit admits at most a fixed finite number $\mathtt{Z}$ of zero-hold options, then moves to the next sibling. Newly created zero-hold work waits for a later visit. Thus an infinite stream of free actions cannot prevent the next sibling's turn. A task owner sets the value scale. A model's own claim that its next step is important cannot increase the owner's weight. Choices with zero estimated gain can be dropped, but the reason is saved for later review.

Value ranks work inside a share, so a high-value claim cannot seize another owner's weight. This can reduce total value. A common pool may rank value across owners, but it has a fixed amount and still checks all caps. Experiments must report that pool separately.

A blocked leaf is skipped for the current round. Positive deficits have a ceiling; negative balances remain as debt. Set each ceiling at least as high as its weighted quantum plus its largest supported hold. Record any credit discarded at that ceiling. Blocked or empty children gain no new quantum. On return, they use their saved balance. Creating empty queues therefore does not mint service credit.

Unused reservation money is returned to cap room on confirmation. For sharing, the difference between the hold and the confirmed cost is credited back in a later round. Unknown calls get no refund yet. An overrun creates sharing debt. Delaying these corrections until the round boundary keeps selection stable while receipts arrive. The policy must cap unresolved exposure per principal, since otherwise a slow-receipt principal can distort the difference between reservation fairness and actual-cost fairness.

D. Local Rate Gates and Progress

Define an optional local rate gate with bucket level b, burst a, and refill rate v. After elapsed time t, refill to $\min(a, b + vt)$. Debit each piece before output and delay delivery until its debt is repaid. Confirmed usage not already paced creates

debt for later calls. Each node owns separate buckets. The scheduler waits for existing debt before holding money, then checks prices and caps at dispatch. Queue removal and hold insertion share one transaction.

No global token-rate theorem is claimed. Each of n nodes has its own rate and burst. A common rate needs shared permits or partitions of a known total. Output slowing does not prove that a provider stopped spending.

If strict holds occupy all remaining room and receipts never arrive, progress stops. This is a real consequence of protecting a finite budget. A system must choose an explicit remedy: obtain a receipt, establish that no charge exists, add money, or move the liability to a funded risk account. Quietly expiring the hold would change the safety claim.

One admission turn

1) Visit the next eligible weighted leaf.
2) Rank its allowed next actions by value per hold.
3) Wait for the local rate gate, then read current prices.
4) In one transaction, check every cap and deficit.
5) Save the call hold; debit its sharing ancestors.
6) Send the call. Keep its hold if cost stays unknown.
7) On receipt, settle cost and queue a share correction.

Fig. 1. Proposed economic scheduling path. The rate gate has local scope.

VI. Analytical Results

A. A Conditional Hard Bound

Assume fixed limits, initially valid exposure, fixed constraint memberships and cohorts, atomic admission and settlement, and $0 \le c_j \le r_j$ for every admitted call. A repeated call key and receipt has no new effect. No unresolved hold is released without a trusted zero-charge result. Then $E_g \le B_g$ for every constraint throughout its cohort. Define latent liability as confirmed charges plus the eventual full cost of every unresolved admitted call. Latent liability is at most exposure, so it too is at most B_g. This statement bounds possible incurred cost, not only the timing of known receipts.

The proof is by induction on state changes. Admission checks the new sum before adding a hold. Moving a hold from active to unknown leaves the sum unchanged. Settlement replaces r_j by c_j, so the sum cannot grow. A trusted zero-charge result reduces it. These are all allowed changes under the assumptions. The argument applies to each intersecting constraint separately; the constraints need not form a tree. Atomic updates make one call's multiple checks refer to a common state. For latent liability, replace each unresolved hold by its eventual cost. Each replacement is no larger under the cost premise. This second inequality is the reason that an exposure bound can imply a bill bound. Without it, bounded ledger values alone say nothing about hidden charges.

This proof does not apply to the baseline heuristic. For any finite hold and any positive remaining budget, an unbounded-cost call can cost more than both. One admitted call is enough to violate the limit. It also fails if an unknown hold expires: an arbitrary sequence of hidden charges can then reuse the same room. No choice of queue order repairs either counterexample.

B. Risk and Overrun Bounds

Suppose risk-mode holds satisfy conditional coverage: before call j is admitted, the chance that $c_j > r_j$, given all prior observations and choices, is at most α_j. Index calls in admission order. Each α_j is chosen before dispatch and may depend on prior observations. Give all calls covered by the claim, including inherited unresolved calls, a total risk allowance α. The sum of assigned allowances must stay at most α on every possible history. Conditional expectation gives `P(any overrun) <= E[sum alpha_j] <= alpha` for a finite campaign, and for a countable campaign by limits of finite prefixes. Thus all covered costs fit their holds with probability at least $1 - \alpha$. On that event the preceding invariant holds. Independence is not required. Valid conditional coverage is required.

Assign risk once per call across all its caps. Delayed receipts do not create new allowance. An inherited call needs its original valid allowance, or a new bound valid given its current history. Missingness can depend on cost, so old unconditional coverage cannot be assumed to hold after selecting only missing receipts. Provider or workload changes can break conditional coverage. A small average error cannot restore that missing premise.

A deterministic statement remains possible without valid probabilities. Let $d_j = \max(0, c_j - r_j)$. If holds stay until settlement and new admission uses the same exposure check, final cost for a constraint is at most its limit plus the sum of d_j for its admitted calls. Each settlement can increase exposure by at most its positive excess; no admission adds excess above the checked limit. Induction gives the result. This bound can be large and is not a hard cap. A per-call error bound ε yields $B_g + N_g\varepsilon$ for N_g calls, not a constant error independent of workload.

C. What Sharing Can Guarantee

Consider one sibling group with zero starting deficits, continuously queued work, no cap blocks, fixed weights, and strictly positive holds at most L. There must be no receipt corrections, deficit clipping, or resets in the interval. Money is integral, so each positive hold is at least one unit and every visit ends. Zero-hold actions are outside this lemma and use the finite visit rule above. Each round adds $Qw_i \ge L$ and serves each child until its next hold does not fit. A ceiling of at least $Qw_i + L$ cannot bind in this setting. After k complete rounds, reservation money is $S_i = kQw_i - D_i$, with $0 \le D_i < L$. Thus normalized service differs by less than $\frac{L}{w_i} + \frac{L}{w_l}$. This is a restricted reservation-money bound, not a latency guarantee.

The full proposed policy can carry initial balances, discard credit, and apply receipt corrections. Those changes add terms to the identity; the restricted bound does not cover them. An exhausted agent cap or unknown receipt can also stop one task while another runs. Under the lemma's assumptions, cumulative absolute cost errors Δ_i and Δ_l add $\frac{\Delta_i}{w_i} + \frac{\Delta_l}{w_l}$ to

its actual-cost bound. Neither result gives equal task success. Evaluation must measure both reserved and confirmed money.

D. Entering the Invariant from Old State

The induction requires a valid starting state. Changing the missing-usage transition for new calls does not supply it. Suppose an old unknown call has lost its hold. Restoring new-call retention while leaving that row at zero permits the same 1,000-to-2,000 trace. An upgrade must first account for every old liability covered by the claim.

If an old call has a validated upper bound, its hold can be restored. Admission then checks the sum of old and new exposure. If that sum already exceeds a cap, new work stops; restoration does not undo existing liability. If only a token estimate or incomplete price data survive, an exact strict hold cannot be inferred. Strict admission against that liability must stay closed until a trusted receipt, a valid bound, or a separately funded risk transfer resolves it. Grandfathering an unbounded old row changes the scope of the claimed guarantee and must be stated.

These rules are model requirements, not an automatic migration policy. Define an estimate-retaining variant that uses the baseline's character-count and 1,024-output estimates but keeps unknown holds. Neither estimate is an enforced billed-token ceiling. Retention repairs room reuse when final cost fits the estimate; it alone does not establish the hard-bound theorem. A second variant assigns confirmed charges to the receipt-time period. It changes the cohort premise and needs a separate analysis across period boundaries. An implementation claim must therefore establish cost bounds, old-state initialization, and cohort membership as separate obligations.

VII. Worked Numerical Example

All numbers in this section are illustrative. An organization has a 100,000 micro-USD cohort limit and 40,000 confirmed cost. Members A and B have 60,000 and 50,000 limits, with 20,000 confirmed for each. A shared agent has a 90,000 limit and the same 40,000 confirmed total. Credits do not bind. The two members have equal sharing weights and a 30,000 quantum. Their first task calls have finite adapter bounds.

The saved schedule charges 2,000,000 micro-USD per million uncached input tokens and 8,000,000 per million output tokens. There is one tier, no cache use, and no reasoning charge. Call A has at most 2,000 input and 2,000 output tokens. Its strict hold is $2,000 \times 2 + 2,000 \times 8 = 20,000$. Call B has at most 3,000 input and 3,000 output tokens. Its hold is $3,000 \times 2 + 3,000 \times 8 = 30,000$.

Both fit their 30,000 deficits. After both admissions, organization exposure is $40,000 + 20,000 + 30,000 = 90,000$. A has 10,000 unused deficit; B has zero. The agent cap is exactly full. Call A then finishes without a receipt. Its 20,000 moves to unknown exposure. No public charge is invented, and no cap room opens.

Call B confirms 3,000 input and 1,500 output tokens. Its cost is $6,000 + 12,000 = 18,000$. Organization confirmed cost becomes 58,000. The 20,000 unknown hold leaves exposure at 78,000. B receives a 12,000 sharing correction at the next round. The agent has only 12,000 room, although the organization has 22,000. This is the effect of intersecting caps.

For A's next step, a large option has hold 15,000 and estimated value 9. A small option has hold 10,000 and value 7. Their value-per-hold scores are 0.0006 and 0.0007. The large option also fails the agent cap. The small option fits A's remaining deficit and every cap, so it is admitted. Exposure becomes 88,000, including 20,000 unknown and 10,000 active holds.

TABLE II
Organization exposure in thousands of micro-USD. Holds include active and unknown calls.

State	Known	Holds	Total
Start	40	0	40
Admit A and B	40	50	90
A unknown; B known	58	20	78
Admit small A	58	30	88
Late A receipt	74	10	84

A late receipt gives the first A call 2,000 input and 1,500 output tokens, costing 16,000. The saved price still applies even if new calls now have a higher price. Confirmed cost becomes 74,000; only the small call's 10,000 hold remains. Exposure falls to 84,000. The released 4,000 can fund later work, subject to the tighter agent cap and sharing turns.

Now change just one assumption: A's first call has no finite bound and later costs 32,000. After the small call was admitted, settlement would make exposure $58,000 + 32,000 + 10,000 = 100,000$. The agent limit of 90,000 is exceeded by 10,000. The earlier hold underestimated cost by 12,000. Stopping new work cannot reverse that bill. This altered trace is why the strict example depends on the adapter bound, not on the arithmetic alone.

VIII. Completed Finite Experiments

The supplied `papers_11_12.py` model is the executable artifact. Its first campaign enumerates every order of two calls with the per-call order admit, unknown, settle. There are 20 such interleavings. Each call has hold 2. The shared cap and each caller cap are 3. Actual costs range independently over integers 0 through 4, giving 25 cost pairs and 500 cases per policy. A blocked call produces no later charge. Every event checks exposure and latent liability against separate inequalities. Duplicate unknown and receipt events check that totals do not change twice.

TABLE III
Completed finite enumeration. Strict cases have both costs at most their holds. Excess means exposure or latent liability exceeds any cap at any event.

Policy	All cases	Strict cases	Strict excess
Release unknown hold	500	180	10
Retain unknown hold	500	180	0

The 180 strict cases per policy use costs 0, 1, or 2. The remaining 320 include costs above the hold. They are delib-

erate premise violations. Retention reaches final shared cost 5 in this outlier set, above the cap 3; release reaches 8. Every retained case satisfies the deterministic overrun inequality. These figures are complete counts in the declared grid, not estimated provider failure rates. They cannot supply risk-mode coverage.

A legacy-state experiment begins with one released unknown hold of 1,000. Changing only future transitions permits a new 1,000 call and final cost 2,000. Restoring a valid old bound blocks the second call and leaves final cost 1,000. Strict initialization rejects missing bounds, unvalidated estimates, and old exposure already above the cap. These cases test the initialization requirement; they do not select a migration rule for a live system.

A separate price experiment enumerates 1,024 vectors: each of five billed classes has count 0 through 3. Two whole-call price tiers depend on total input count. Integer rounding gives an upper hold of 67 synthetic money units. Every vector fits that hold. Replacing the live schedule leaves the saved schedule unchanged. This tests tier search, all five classes, and price snapshots within a finite box. It does not establish that a provider respects the box or that an invoice uses the saved rates.

For sharing, seeds 0 through 29 each drive 40 complete rounds with weights 1, 2, and 3, base quantum 10, and positive holds from 1 through 10. All 1,200 rounds satisfy the exact service identity and the stated normalized-gap bound. The largest observed normalized gap is 9 units. A separate clipping case breaks the identity when discarded credit is omitted. An endless zero-cost source admits exactly three actions under visit limit three, then yields the turn. This is a test of the restricted lemma and finite visit rule. It includes no cap blocks, refunds, delayed receipts, task values, or full sharing tree.

The randomized ledger campaign adds 200 events for each of the same 30 seeds. Its hidden costs fit their holds by construction. It checks every matching cap, idempotent settlement, and final cost conservation. Worked examples add the 1,000-to-2,000 trace and the tiered-price trace. Across these campaigns the artifact performs 84,849 assertions. The count checks reproducibility; neither it nor the number of generated events is an independent sample size.

IX. Evaluation Protocol

The study should test useful work per budget, fair sharing, and exposure hidden by late receipts. An event simulator tests policy choices. A transactional database harness tests concurrent accounting. Neither has been run for this paper.

Each simulated task supplies a finite sequence of possible actions, value outcomes, token vectors, and receipt times. Start with synthetic families so all ground truth is known. Include short independent jobs, long chains whose later value depends on earlier success, and delegated trees with a fixed root weight. Vary task count, depth, model price ratios, input-tier thresholds, and the fraction of unknown receipts. Use both bounded costs and unbounded synthetic tails. The latter cases test failure reporting and risk exposure, not a theorem that assumes bounded cost.

Vary receipt delay independently from provider completion, then correlate it with high cost. Include missing and duplicate receipts and price changes. Keep true provider cost in the simulator even when the scheduler cannot see it. Otherwise, releasing unknown holds may look efficient simply because the metric misses the bills.

Compare five policies under the same arrivals and outcomes: the defined estimate-and-release baseline with first-come order; confirmed-cost-only admission; a plain escrow ledger with strict upper holds and first-come order; weighted sharing with rough holds; and the full proposed policy. Add an offline oracle with complete cost and value knowledge as an upper reference, clearly marked as unattainable online. All policies must have the same provider options, caps, and retry rules. Include a separate comparison with local token pacing enabled to avoid attributing output delay to the money scheduler.

Report total task value, completed tasks, deadline misses, and unused budget. Report peak actual liability, maximum cap excess, time spent blocked by unknown holds, and money assigned to the risk account. For sharing, show each principal's reserved and confirmed money divided by its weight, plus the largest pairwise gap among comparable eligible principals. Report ineligible time separately. A single global fairness score would hide a member blocked by a distinct agent cap.

Ablations remove unknown holds, price snapshots, parent weight conservation, and settlement corrections one at a time. A tier-boundary workload tests the envelope calculation. A queue-splitting workload tests whether more child IDs gain a larger parent share. A model-value error workload changes the score quality while keeping costs fixed. This shows whether a gain comes from allocation or from an unrealistically accurate value predictor.

Publish the grid and use at least 30 independent seeds per stochastic case, paired across policies. Report paired differences with bootstrap confidence intervals over runs, quantiles, and deterministic counterexamples. Separate workload and option-selection randomness. Calls within a run are not independent observations. Choose primary metrics before results, and report where strict holds finish less useful work.

The integration harness should use two or more processes with barriers around admission and settlement. Test overlapping scope caps, a new limit during active work, a late receipt after a period boundary, and an organization shared by nodes. Assert the ledger identities after each event. For local token buckets, explicitly show that two nodes do not share one bucket. Shared state in that test would violate the specified local-gate model.

X. Reproducible State and Fixtures

For replay, save each call's identifier, cohort, memberships, owner path, price, envelope, hold, status, and nullable confirmed counts. Save cap limits, sharing weights and deficits, and local bucket levels. Apply each event atomically and emit totals. Sort equal-time events by identifier. Repeating an

admission identifier returns its prior record; changed input is rejected. These records fully define the synthetic replay.

Four small synthetic fixtures check distinct obligations. First, replay the 1,000-unit baseline counterexample. The baseline ends at 2,000. With retained exposure, the second admission is blocked while the first remains unknown. After the first confirms at 1,000, that cap is spent and the second still cannot start. A duplicate receipt must leave both outcomes unchanged.

Second, let a schedule charge one micro-USD per input token up to 1,000 tokens and three above that threshold, with no other charge. An 800-token estimate reserves 800, but actual input of 1,200 costs 3,600. An enforced input bound of 2,000 requires a 6,000 hold under this schedule. Changing the upper rate to four after admission must not change the saved call's 3,600 settlement. This fixture separates a price snapshot from an input-tier bound.

Third, let two calls each need 10,000 while their shared agent cap has only 10,000 room. Concurrent admission may select either call but must admit exactly one. Repeat with different owners that share that agent constraint. Owner-only locks can let both calls read the same room. Locking the shared cap during admission prevents that race.

Fourth, test the sharing lemma with $Q = L = 10$ and holds of size 6. With a clipping ceiling of 10, two rounds admit 12 and leave deficit 4; the unclipped expression would wrongly predict 16. With a ceiling of 20 and zero initial deficit, round one admits 6 and leaves 4; round two admits 12 more and leaves 2. Total service is 18, equal to $2 \times 10 - 2$. The latter run satisfies the lemma's premises. The finite artifact executes these calculations. Its two-order admission check models atomic events; it does not run concurrent database processes.

A study must publish event inputs, prices, bounds, cap memberships, weights, tie order, and receipt rules. Save latent costs apart from visible receipts. Check exposure conservation, receipt deduplication, price immutability, and parent weight conservation after every event. Record discarded sharing credit and debt corrections so a run outside the lemma cannot be presented as proof of its bound.

XI. Limitations and Conclusion

Strict holds can block useful work. Value estimates can favor cheap but poor answers. Weights are owner policy. Delegation identity must be stable or queue splitting can defeat sharing. No spend guarantee covers an unknown external price or an unbounded provider charge. Durable accounting alone does not solve these problems.

Agent budgets need rules for both future cost and competing useful work. The method retains unknown exposure, checks all caps, and shares admission through a fixed-weight tree. Hard bounds need finite cost envelopes; risk bounds need conditional coverage. The example shows how these distinctions change admission. The finite study tests the accounting invariant, its outlier bound, the need to cover old liabilities, and a restricted sharing identity. A full evaluation must still measure how these controls affect useful work under real delayed receipts. The distinction between a safe ledger and a validated provider-cost bound remains central.

References

[1] P. E. O'Neil, "The Escrow Transactional Method," *ACM Transactions on Database Systems*, vol. 11, no. 4, pp. 405–430, 1986, doi: 10.1145/7239.7265.

[2] V. Balegas *et al.*, "Extending Eventually Consistent Cloud Databases for Enforcing Numeric Invariants," in *2015 IEEE 34th Symposium on Reliable Distributed Systems*, IEEE, 2015, pp. 31–36. doi: 10.1109/SRDS.2015.32.

[3] G. Varghese and M. Shreedhar, "Efficient Fair Queueing using Deficit Round Robin," WUCS-94–17, 1994. doi: 10.7936/K7513WG9.

[4] I. Stoica, H. Zhang, and T. S. E. Ng, "A Hierarchical Fair Service Curve Algorithm for Link-Sharing, Real-Time and Priority Services," in *Proceedings of ACM SIGCOMM*, 1997, pp. 249–262. [Online]. Available: https://conferences.sigcomm.org/sigcomm/1997/papers/p011.pdf

[5] A. Ghodsi, M. Zaharia, B. Hindman, A. Konwinski, S. Shenker, and I. Stoica, "Dominant Resource Fairness: Fair Allocation of Multiple Resource Types," in *8th USENIX Symposium on Networked Systems Design and Implementation*, USENIX Association, 2011. [Online]. Available: https://www.usenix.org/conference/nsdi11/dominant-resource-fairness-fair-allocation-multiple-resource-types

[6] F. P. Kelly, A. K. Maulloo, and D. K. H. Tan, "Rate Control for Communication Networks: Shadow Prices, Proportional Fairness and Stability," *Journal of the Operational Research Society*, vol. 49, no. 3, pp. 237–252, 1998, [Online]. Available: https://www.statslab.cam.ac.uk/~frank/rate.pdf

[7] D. Bertsimas and M. Sim, "The Price of Robustness," *Operations Research*, vol. 52, no. 1, pp. 35–53, 2004, doi: 10.1287/opre.1030.0065.

[8] A. Badanidiyuru, R. Kleinberg, and A. Slivkins, "Bandits with Knapsacks," in *2013 IEEE 54th Annual Symposium on Foundations of Computer Science*, 2013. doi: 10.1109/FOCS.2013.30.

[9] J. Heinanen and R. Guerin, "A Single Rate Three Color Marker," technical report 2697, Sep. 1999. doi: 10.17487/RFC2697.